

Interval width and bootstrap iterations for conditional infit MSQ

Simulation material for a follow-up to Johansson (2025)

Magnus Johansson

2026-07-29

Table of contents

Scope	1
Extending a run	1
Resumable runner	2
Data generation	2
Study 1: convergence of the interval	3
Median interval width by B and width	4
Relative to the B = 10 000 reference	5
Draws expected in each tail	5
Study 2 and 3: error rates and multiplicity	6
Family-wise false positive rate under a true model ($k = 0$)	7
Detection and false positives ($k = 3$)	8
Expected flags per analysis ($k = 3$)	10
Direction of false flags under a true model	10
Study 3: FWER vs BH	11
Per-item rates	11
Family-wise error rate under a true model	13
Realised false discovery proportion ($k = 3$)	13
The BH reachability floor	14
Notes	15

Scope

Simulation material only — no manuscript text. Three questions:

- **Study 1 (convergence).** The HDCI endpoint at width w is an extreme order statistic, so its stability is governed by the expected number of bootstrap draws in each tail, $(1 - w)/2 * B$. Where does the interval stop changing, for each width, up to $B = 10\,000$?
- **Study 2 (error rates).** Detection and false positive rates over the full (w, B) grid.
- **Study 3 (multiplicity).** Marginal, FWER (Westfall-Young step-down) and BH p-values, read off the same bootstraps as Study 2.

Every (rep, k, n, B) cell uses an **independent** bootstrap; no draws are reused across B conditions.

Extending a run

Replication counts are set by `R1` and `R2` below. Runs are **resumable**: raise `R2` from 250 to 500, re-render, and only replications 251-500 are computed — the cached cells are reused. Seeds are a deterministic function of (rep, k, n, B) , so a resumed run is identical to having run the larger number from the start. Results are checkpointed to disk after every chunk, so an interrupted run also resumes from where it stopped rather than from zero.

Delete the `.rds` files to force a full re-run.

```
suppressMessages({
  library(easyRasch2)
  library(ggdist)
  library(eRm)
  library(MASS)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})

WIDTHS <- c(0.90, 0.95, 0.99, 0.995, 0.999)
```

```

NS      <- c(150, 250, 500, 1000)
NCORES  <- 10
ALPHA   <- 0.05
CHUNK   <- 100          # cells per checkpoint

B1 <- c(100, 200, 400, 1000, 2000, 5000, 10000)
R1 <- 30
RES_1 <- "infit_width_iterations_S1.rds"

B2 <- c(100, 200, 400, 1000, 2000)
R2 <- 250          # raise to 500 and re-render to extend
NMISFITS <- c(0, 1, 3)
RES_2 <- "infit_width_iterations_S2.rds"

```

Resumable runner

```

run_resumable <- function(res_file, grid, keys, fn,
                          ncores = NCORES, chunk_size = CHUNK,
                          extra = list()) {
  key_of <- function(g) {
    if (nrow(g) == 0L) return(character(0))
    do.call(paste, c(unname(as.list(g[keys])), sep = "|"))
  }
  if (file.exists(res_file)) {
    acc <- readRDS(res_file)
  } else {
    acc <- c(list(res = list(), grid = grid[0, , drop = FALSE],
                  elapsed_min = 0), extra)
  }
  todo <- which(!(key_of(grid) %in% key_of(acc$grid)))
  if (length(todo) == 0L) {
    message("cached: ", nrow(acc$grid), " cells, nothing to run")
    return(acc)
  }
  message("running ", length(todo), " new cells (",
          nrow(acc$grid), " already cached)")
  for (s in seq(1, length(todo), by = chunk_size)) {
    idx <- todo[s:min(s + chunk_size - 1L, length(todo))]
    t0 <- Sys.time()
    out <- mclapply(idx, function(i) fn(grid[i, , drop = FALSE])),
              mc.cores = ncores)
    acc$res <- c(acc$res, out)
    acc$grid <- rbind(acc$grid, grid[idx, , drop = FALSE])
    acc$elapsed_min <- acc$elapsed_min +
      as.numeric(Sys.time() - t0, "mins")
    acc <- utils::modifyList(acc, extra)
    saveRDS(acc, res_file)
    message(" checkpoint ", nrow(acc$grid), " / ", nrow(grid),
            " (", round(acc$elapsed_min, 1), " min)")
  }
  acc
}

```

Data generation

Follows Johansson (2025): 20 dichotomous items, locations `runif(20, -2, 2)`, person locations bivariate normal with SD 1.5 and inter-dimensional correlation 0.15, misfit induced by loading items on a second dimension, misfitting items at approximately 0, -1 and -2 logits.

```

make_master <- function(n_misfit, N = 10000, seed = 20250727) {
  set.seed(seed)
  locs <- runif(20, -2, 2)
  misfit_idx <- integer(0)

```

```

for (tg in c(0, -1, -2)) {
  cand <- setdiff(order(abs(locs - tg)), misfit_idx)[1]
  misfit_idx <- c(misfit_idx, cand)
}
use <- misfit_idx[seq_len(n_misfit)]
W <- cbind(rep(1, 20), rep(0, 20))
if (n_misfit > 0) W[use, ] <- matrix(c(0, 1), nrow = length(use), ncol = 2,
                                     byrow = TRUE)

Sigma <- matrix(c(1.5^2, .15 * 1.5^2, .15 * 1.5^2, 1.5^2), 2, 2)
set.seed(seed + n_misfit)
th <- MASS::mvrnorm(N, mu = c(0, 0), Sigma = Sigma)
d <- as.data.frame(eRm::sim.xdim(persons = th, items = locs,
                                Sigma = Sigma, weightmat = W))

names(d) <- paste0("V", 1:20)
list(data = d, locs = locs, misfit = use)
}

masters <- lapply(NMISFITS, make_master)
names(masters) <- as.character(NMISFITS)
misfit_all <- masters[["3"]]$misfit
locs <- masters[["3"]]$locs

data.frame(item = paste0("V", misfit_all), intended = c(0, -1, -2),
           actual = round(locs[misfit_all], 2))

```

item	intended	actual
V8	0	0.04
V12	-1	-1.00
V11	-2	-2.00

```

bounds_at <- function(results, width, items) {
  vapply(items, function(it) {
    v <- results$InfitMSQ[results$Item == it]
    as.numeric(ggdist::hdci(v, .width = width))[1:2]
  }, numeric(2))
}

# 0 = not flagged, 1 = overfit (below interval), 2 = underfit (above)
flag_code <- function(msq, lo, hi) {
  ifelse(msq > hi, 2L, ifelse(msq < lo, 1L, 0L))
}

```

Study 1: convergence of the interval

Null data only ($k = 0$), independent bootstrap per B.

```

one_rep_S1 <- function(g) {
  rep <- g$rep; n <- g$n
  set.seed(5e7 + 1e3 * n + rep)
  d <- masters[["0"]]$data[sample(10000, n), ]
  items <- names(d)
  out <- vector("list", length(B1))
  for (j in seq_along(B1)) {
    co <- try(RMitemInfitCutoff(d, iterations = B1[j], parallel = FALSE,
                              seed = rep * 100 + j, verbose = FALSE),
             silent = TRUE)
    if (inherits(co, "try-error")) next
    out[[j]] <- do.call(rbind, lapply(WIDTHS, function(w) {
      b <- bounds_at(co$results, w, items)
      data.frame(B = B1[j], w = w, item = seq_along(items),

```

```

        width = b[2, ] - b[1, ])
    )))
  }
  do.call(rbind, out)
}

grid1 <- expand.grid(rep = 1:R1, n = NS)
s1 <- run_resumable(RES_1, grid1, c("rep", "n"), one_rep_S1)

long1 <- do.call(rbind, lapply(seq_len(nrow(s1$grid)), function(i) {
  r <- s1$res[[i]]
  if (is.null(r)) return(NULL)
  cbind(rep = s1$grid$rep[i], n = s1$grid$n[i], r)
})))
cat("Study 1:", nrow(s1$grid), "cells |", round(s1$elapsed_min, 1), "min\n")

```

Study 1: 120 cells | 76.5 min

Median interval width by B and width

```

long1 |>
  group_by(n, w, B) |>
  summarise(v = median(width), .groups = "drop") |>
  pivot_wider(names_from = B, values_from = v, names_prefix = "B=") |>
  mutate(across(where(is.numeric), \(x) round(x, 3))) |>
  arrange(n, w)

```

n	w	B=100	B=200	B=400	B=1000	B=2000	B=5000	B=10000
150	0.900	0.311	0.317	0.318	0.320	0.322	0.323	0.323
150	0.950	0.371	0.378	0.379	0.382	0.384	0.385	0.385
150	0.990	0.470	0.484	0.494	0.500	0.505	0.507	0.506
150	0.995	0.494	0.519	0.535	0.545	0.551	0.553	0.552
150	0.999	0.494	0.542	0.591	0.620	0.640	0.645	0.649
250	0.900	0.237	0.241	0.244	0.245	0.246	0.247	0.247
250	0.950	0.282	0.287	0.291	0.292	0.294	0.295	0.295
250	0.990	0.356	0.371	0.379	0.383	0.384	0.387	0.388
250	0.995	0.373	0.397	0.409	0.416	0.417	0.421	0.423
250	0.999	0.373	0.414	0.448	0.473	0.482	0.493	0.495
500	0.900	0.167	0.171	0.173	0.174	0.174	0.174	0.174
500	0.950	0.201	0.203	0.206	0.207	0.206	0.208	0.208
500	0.990	0.252	0.264	0.267	0.271	0.272	0.273	0.273
500	0.995	0.265	0.282	0.285	0.294	0.296	0.297	0.297
500	0.999	0.265	0.294	0.314	0.335	0.343	0.346	0.347
1000	0.900	0.120	0.121	0.122	0.122	0.123	0.123	0.123
1000	0.950	0.142	0.143	0.145	0.146	0.147	0.147	0.147
1000	0.990	0.181	0.186	0.190	0.191	0.192	0.192	0.193
1000	0.995	0.189	0.197	0.203	0.207	0.210	0.210	0.210
1000	0.999	0.189	0.205	0.223	0.236	0.241	0.244	0.245

Relative to the B = 10 000 reference

Values below 1 mean the interval is too **narrow**, which inflates the false positive rate.

```
long1 |>
  group_by(n, w, B) |>
  summarise(v = median(width), .groups = "drop") |>
  group_by(n, w) |>
  mutate(rel = round(v / v[B == max(B)], 3)) |>
  select(-v) |>
  pivot_wider(names_from = B, values_from = rel, names_prefix = "B=") |>
  arrange(n, w)
```

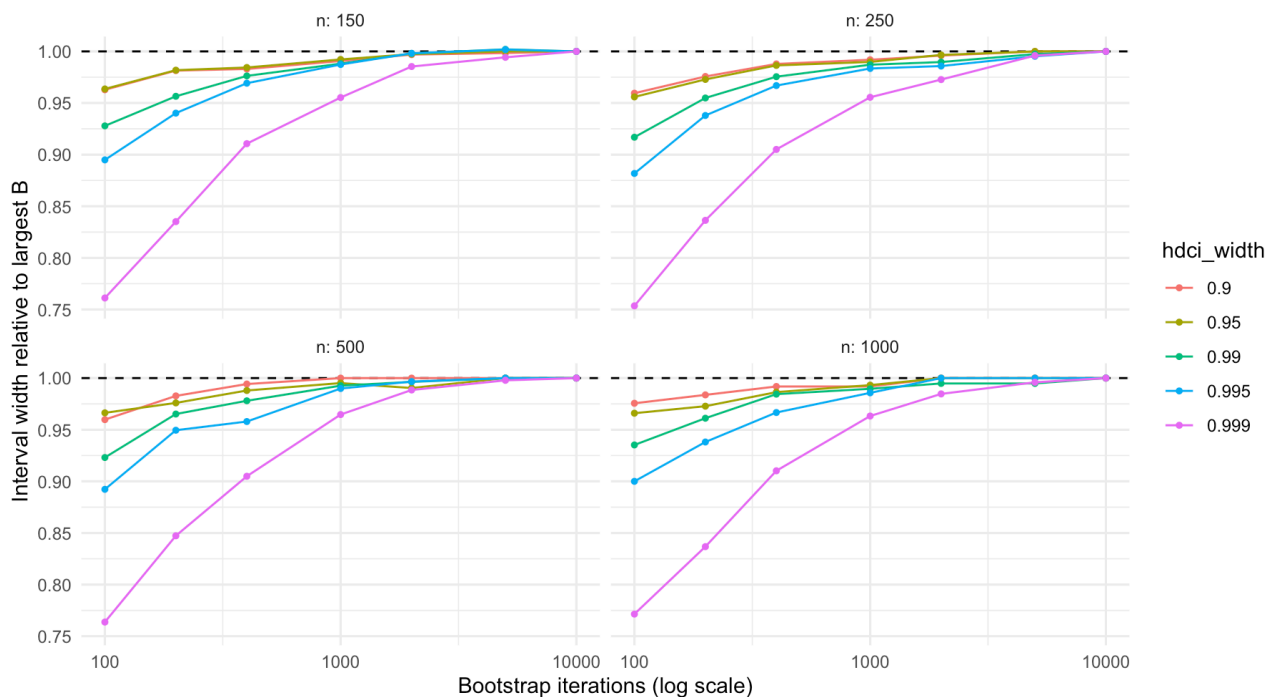
n	w	B=100	B=200	B=400	B=1000	B=2000	B=5000	B=10000
150	0.900	0.963	0.981	0.983	0.991	0.997	0.998	1
150	0.950	0.964	0.982	0.984	0.992	0.997	1.000	1
150	0.990	0.928	0.957	0.976	0.988	0.998	1.002	1
150	0.995	0.895	0.940	0.969	0.987	0.998	1.002	1
150	0.999	0.761	0.835	0.911	0.955	0.985	0.994	1
250	0.900	0.960	0.976	0.988	0.992	0.996	1.000	1
250	0.950	0.956	0.973	0.986	0.990	0.997	1.000	1
250	0.990	0.917	0.955	0.976	0.987	0.990	0.997	1
250	0.995	0.882	0.938	0.967	0.983	0.986	0.995	1
250	0.999	0.754	0.836	0.905	0.956	0.973	0.996	1
500	0.900	0.960	0.983	0.994	1.000	1.000	1.000	1
500	0.950	0.966	0.976	0.988	0.995	0.990	1.000	1
500	0.990	0.923	0.965	0.978	0.993	0.996	1.000	1
500	0.995	0.892	0.949	0.958	0.990	0.997	1.000	1
500	0.999	0.764	0.847	0.905	0.965	0.988	0.998	1
1000	0.900	0.976	0.984	0.992	0.992	1.000	1.000	1
1000	0.950	0.966	0.973	0.986	0.993	1.000	1.000	1
1000	0.990	0.935	0.961	0.984	0.990	0.995	0.995	1
1000	0.995	0.900	0.938	0.967	0.986	1.000	1.000	1
1000	0.999	0.771	0.837	0.910	0.963	0.985	0.996	1

Draws expected in each tail

```
expand.grid(w = WIDTHS, B = B1) |>
  mutate(per_tail = (1 - w) / 2 * B) |>
  pivot_wider(names_from = B, values_from = per_tail, names_prefix = "B=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1)))
```

w	B=100	B=200	B=400	B=1000	B=2000	B=5000	B=10000
0.9	5.0	10.0	20.0	50.0	100	250.0	500
0.9	2.5	5.0	10.0	25.0	50	125.0	250
1.0	0.5	1.0	2.0	5.0	10	25.0	50
1.0	0.3	0.5	1.0	2.5	5	12.5	25
1.0	0.1	0.1	0.2	0.5	1	2.5	5

```
long1 |>
  group_by(n, w, B) |>
  summarise(v = median(width), .groups = "drop") |>
  group_by(n, w) |>
  mutate(rel = v / v[B == max(B)]) |>
  ggplot(aes(B, rel, colour = factor(w))) +
  geom_hline(yintercept = 1, linetype = 2) +
  geom_line() + geom_point(size = 1) +
  scale_x_log10() +
  facet_wrap(~n, labeller = label_both) +
  labs(x = "Bootstrap iterations (log scale)",
       y = "Interval width relative to largest B",
       colour = "hdci_width") +
  theme_minimal(base_size = 11)
```



Study 2 and 3: error rates and multiplicity

One pass produces both: HDCl flags at every width, and the three p-value rules.

```
one_rep_S2 <- function(g) {
  rep <- g$rep; k <- g$k; n <- g$n
  set.seed(6e7 + 1e6 * k + 1e3 * n + rep)
  d <- masters[[as.character(k)]]$data[sample(10000, n), ]
  items <- names(d)
  obs <- RMitemInfit(d, output = "dataframe")
  msq <- obs$Infit_MSQ[match(items, obs$Item)]
  hd <- vector("list", length(B2)); pv <- vector("list", length(B2))
  for (j in seq_along(B2)) {
    co <- try(RMitemInfitCutoff(d, iterations = B2[j], parallel = FALSE,
                               seed = rep * 100 + j, verbose = FALSE),
              silent = TRUE)
    if (inherits(co, "try-error")) next
    hd[[j]] <- do.call(rbind, lapply(WIDTHS, function(w) {
      b <- bounds_at(co$results, w, items)
      data.frame(B = B2[j], w = w, item = seq_along(items),
                 code = flag_code(msq, b[1, ], b[2, ]))
    })))
  }
  fw <- try(suppressWarnings(
```

```

      RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fwer",
        alpha = ALPHA, output = "dataframe")), silent = TRUE)
bh <- try(suppressWarnings(
  RMitemInfit(d, cutoff = co, p_value = TRUE, correction = "fdr_bh",
    alpha = ALPHA, output = "dataframe")), silent = TRUE)
if (!inherits(fw, "try-error") && !inherits(bh, "try-error")) {
  i1 <- match(items, fw$Item); i2 <- match(items, bh$Item)
  pv[[j]] <- data.frame(B = B2[j], item = seq_along(items),
    p_marg = fw$p_infit[i1],
    padj_fwer = fw$padj_infit[i1],
    padj_bh = bh$padj_infit[i2])
}
}
list(hdci = do.call(rbind, hd), pval = do.call(rbind, pv))
}

grid2 <- expand.grid(rep = 1:R2, n = NS, k = NMISFITS)
s2 <- run_resumable(RES_2, grid2, c("rep", "k", "n"), one_rep_S2,
  extra = list(misfit = misfit_all))

bind_part <- function(part) {
  do.call(rbind, lapply(seq_len(nrow(s2$grid)), function(i) {
    r <- s2$res[[i]][[part]]
    if (is.null(r)) return(NULL)
    cbind(rep = s2$grid$rep[i], k = s2$grid$k[i], n = s2$grid$n[i], r)
  })))
}
tag <- function(x) x |>
  mutate(is_misfit = mapapply(function(k, it) it %in% s2$misfit[seq_len(k)],
    k, item))
hdci2 <- bind_part("hdci") |> tag() |> mutate(flag = as.integer(code > 0))
pval2 <- bind_part("pval") |> tag()
cat("Study 2/3:", nrow(s2$grid), "cells |", round(s2$elapsed_min, 1), "min |",
  length(unique(s2$grid$rep)), "reps\n")

```

Study 2/3: 3000 cells | 382.2 min | 250 reps

Family-wise false positive rate under a true model ($k = 0$)

```

hdci2 |>
  filter(k == 0) |>
  group_by(w, B, n, rep) |>
  summarise(any = as.integer(sum(flag) > 0), .groups = "drop") |>
  group_by(w, B, n) |>
  summarise(v = 100 * mean(any), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(w, B)

```

w	B	n=150	n=250	n=500	n=1000
0.9	100	92.0	91.6	93.6	89.6
0.9	100	72.0	76.4	77.6	70.4
0.9	200	89.2	88.4	90.4	86.4
0.9	200	64.8	69.6	69.2	63.2
0.9	400	89.2	87.6	88.8	87.2
0.9	400	64.0	67.6	67.6	66.0
0.9	1000	89.2	84.4	90.4	83.2

w	B	n=150	n=250	n=500	n=1000
0.9	1000	60.8	65.2	66.0	59.6
0.9	2000	88.0	84.4	90.4	86.0
0.9	2000	61.6	64.4	63.6	58.0
1.0	100	31.6	40.8	35.2	35.6
1.0	100	25.2	34.8	30.0	28.8
1.0	100	25.2	34.8	30.0	28.8
1.0	200	27.2	33.6	26.8	31.2
1.0	200	18.8	23.2	18.8	22.4
1.0	200	14.4	20.0	16.4	18.0
1.0	400	21.2	24.4	23.2	24.4
1.0	400	12.4	13.2	15.6	15.6
1.0	400	7.2	8.8	8.4	10.0
1.0	1000	19.2	22.0	20.0	22.8
1.0	1000	10.4	12.0	12.4	14.0
1.0	1000	2.8	5.2	4.4	6.0
1.0	2000	15.2	20.4	19.6	18.8
1.0	2000	7.2	11.6	10.4	12.0
1.0	2000	2.4	2.8	2.0	4.4

Detection and false positives (k = 3)

```
hdc12 |>
  filter(k == 3) |>
  mutate(type = ifelse(is_misfit, "detection", "false_pos")) |>
  group_by(w, B, n, type) |>
  summarise(v = 100 * mean(flag), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(type, w, B)
```

w	B	type	n=150	n=250	n=500	n=1000
0.9	100	detection	96.0	99.5	100.0	100.0
0.9	100	detection	92.5	98.4	100.0	100.0
0.9	200	detection	96.4	99.3	100.0	100.0
0.9	200	detection	94.0	98.0	100.0	100.0
0.9	400	detection	96.4	99.3	100.0	100.0
0.9	400	detection	93.2	98.5	99.9	100.0
0.9	1000	detection	96.0	99.2	100.0	100.0
0.9	1000	detection	93.9	98.1	100.0	100.0
0.9	2000	detection	95.9	99.3	100.0	100.0
0.9	2000	detection	94.1	98.4	100.0	100.0
1.0	100	detection	84.4	95.6	99.6	100.0
1.0	100	detection	80.8	92.9	99.5	100.0

w	B	type	n=150	n=250	n=500	n=1000
1.0	100	detection	80.8	92.9	99.5	100.0
1.0	200	detection	83.1	94.7	99.7	100.0
1.0	200	detection	78.9	92.8	99.7	100.0
1.0	200	detection	75.2	91.1	99.7	100.0
1.0	400	detection	82.1	93.3	99.7	100.0
1.0	400	detection	76.8	91.3	99.7	100.0
1.0	400	detection	68.7	87.2	99.1	100.0
1.0	1000	detection	82.5	93.9	99.7	100.0
1.0	1000	detection	76.1	91.6	99.7	100.0
1.0	1000	detection	65.3	84.5	99.1	100.0
1.0	2000	detection	81.5	94.0	99.7	100.0
1.0	2000	detection	76.0	91.2	99.7	100.0
1.0	2000	detection	64.9	85.1	99.5	100.0
0.9	100	false_pos	19.6	27.1	42.9	65.4
0.9	100	false_pos	12.2	18.3	32.5	54.4
0.9	200	false_pos	19.1	25.4	41.3	65.0
0.9	200	false_pos	10.8	17.1	30.4	53.6
0.9	400	false_pos	17.5	25.0	41.5	65.0
0.9	400	false_pos	10.1	16.6	29.9	53.0
0.9	1000	false_pos	16.8	24.0	40.2	64.3
0.9	1000	false_pos	9.3	15.6	29.2	52.1
0.9	2000	false_pos	16.3	23.7	40.0	64.0
0.9	2000	false_pos	9.3	15.3	28.1	52.0
1.0	100	false_pos	4.6	8.8	16.8	36.8
1.0	100	false_pos	4.0	7.3	14.4	33.1
1.0	100	false_pos	4.0	7.3	14.4	33.1
1.0	200	false_pos	3.4	6.6	14.6	32.3
1.0	200	false_pos	2.6	4.8	10.8	27.2
1.0	200	false_pos	2.2	4.3	9.2	24.1
1.0	400	false_pos	2.8	5.4	13.0	31.5
1.0	400	false_pos	2.0	4.0	9.7	26.0
1.0	400	false_pos	1.3	2.4	6.2	18.6
1.0	1000	false_pos	2.6	4.9	11.8	29.8
1.0	1000	false_pos	1.3	3.0	7.7	23.2
1.0	1000	false_pos	0.5	1.2	3.9	13.2
1.0	2000	false_pos	2.1	4.8	11.0	29.2
1.0	2000	false_pos	1.4	2.8	7.3	22.4
1.0	2000	false_pos	0.4	1.1	3.0	12.0

Expected flags per analysis (k = 3)

```
hdci2 |>
  filter(k == 3) |>
  group_by(w, B, n, rep) |>
  summarise(t = sum(flag[is_misfit]), f = sum(flag[!is_misfit]),
    .groups = "drop") |>
  group_by(w, B, n) |>
  summarise(true = round(mean(t), 2), false = round(mean(f), 2),
    .groups = "drop") |>
  pivot_wider(names_from = n, values_from = c(true, false)) |>
  arrange(w, B)
```

w	B	true_150	true_250	true_500	true_1000	false_150	false_250	false_500	false_1000
0.900	100	2.88	2.98	3.00	3	3.33	4.60	7.30	11.12
0.900	200	2.89	2.98	3.00	3	3.25	4.31	7.02	11.04
0.900	400	2.89	2.98	3.00	3	2.98	4.26	7.05	11.06
0.900	1000	2.88	2.98	3.00	3	2.85	4.09	6.84	10.93
0.900	2000	2.88	2.98	3.00	3	2.76	4.03	6.80	10.87
0.950	100	2.78	2.95	3.00	3	2.07	3.12	5.52	9.26
0.950	200	2.82	2.94	3.00	3	1.83	2.91	5.17	9.10
0.950	400	2.80	2.96	3.00	3	1.71	2.82	5.08	9.00
0.950	1000	2.82	2.94	3.00	3	1.59	2.66	4.96	8.86
0.950	2000	2.82	2.95	3.00	3	1.58	2.60	4.77	8.83
0.990	100	2.53	2.87	2.99	3	0.79	1.50	2.86	6.26
0.990	200	2.49	2.84	2.99	3	0.58	1.13	2.48	5.50
0.990	400	2.46	2.80	2.99	3	0.48	0.92	2.21	5.36
0.990	1000	2.48	2.82	2.99	3	0.44	0.83	2.00	5.07
0.990	2000	2.44	2.82	2.99	3	0.36	0.81	1.87	4.97
0.995	100	2.42	2.79	2.98	3	0.67	1.25	2.45	5.63
0.995	200	2.37	2.78	2.99	3	0.44	0.81	1.83	4.63
0.995	400	2.30	2.74	2.99	3	0.33	0.68	1.65	4.42
0.995	1000	2.28	2.75	2.99	3	0.23	0.51	1.30	3.94
0.995	2000	2.28	2.74	2.99	3	0.24	0.47	1.24	3.82
0.999	100	2.42	2.79	2.98	3	0.67	1.25	2.45	5.63
0.999	200	2.26	2.73	2.99	3	0.38	0.72	1.56	4.10
0.999	400	2.06	2.62	2.97	3	0.22	0.40	1.05	3.16
0.999	1000	1.96	2.54	2.97	3	0.08	0.21	0.67	2.25
0.999	2000	1.95	2.55	2.98	3	0.07	0.19	0.52	2.04

Direction of false flags under a true model

```
hdci2 |>
  filter(k == 0, code > 0) |>
  mutate(dir = ifelse(code == 1, "overfit", "underfit")) |>
  group_by(w, B, dir) |>
  summarise(ct = n(), .groups = "drop") |>
  group_by(w, B) |>
```

```
mutate(pct = round(100 * ct / sum(ct), 1)) |>
ungroup() |> select(-ct) |>
pivot_wider(names_from = dir, values_from = pct, values_fill = 0)
```

w	B	overfit	underfit
0.900	100	49.1	50.9
0.900	200	49.2	50.8
0.900	400	48.6	51.4
0.900	1000	48.0	52.0
0.900	2000	47.9	52.1
0.950	100	50.4	49.6
0.950	200	50.3	49.7
0.950	400	50.8	49.2
0.950	1000	49.5	50.5
0.950	2000	48.2	51.8
0.990	100	54.4	45.6
0.990	200	52.8	47.2
0.990	400	52.7	47.3
0.990	1000	48.9	51.1
0.990	2000	48.3	51.7
0.995	100	55.2	44.8
0.995	200	54.2	45.8
0.995	400	52.3	47.7
0.995	1000	45.3	54.7
0.995	2000	47.7	52.3
0.999	100	55.2	44.8
0.999	200	57.1	42.9
0.999	400	53.8	46.2
0.999	1000	50.0	50.0
0.999	2000	50.0	50.0

Study 3: FWER vs BH

Per-item rates

```
pv_long <- pval2 |>
mutate(fwer = as.integer(padj_fwer < ALPHA),
       bh = as.integer(padj_bh < ALPHA),
       uncorr = as.integer(p_marg < ALPHA)) |>
pivot_longer(c(fwer, bh, uncorr), names_to = "rule", values_to = "flag")

pv_long |>
filter(k %in% c(0, 3)) |>
mutate(type = case_when(k == 0 ~ "type_I", is_misfit ~ "detection",
                       TRUE ~ "false_pos")) |>
group_by(rule, B, n, type) |>
summarise(v = 100 * mean(flag, na.rm = TRUE), .groups = "drop") |>
```

```

pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
mutate(across(where(is.numeric), \(x) round(x, 1))) |>
arrange(type, rule, B)

```

rule	B	type	n=150	n=250	n=500	n=1000
bh	100	detection	13.6	35.5	80.1	99.6
bh	200	detection	70.9	90.7	99.9	100.0
bh	400	detection	77.6	92.8	99.7	100.0
bh	1000	detection	78.9	93.3	99.7	100.0
bh	2000	detection	80.0	93.7	99.9	100.0
fwer	100	detection	74.4	90.5	99.6	100.0
fwer	200	detection	74.8	89.6	99.6	100.0
fwer	400	detection	73.9	90.8	99.6	100.0
fwer	1000	detection	73.7	89.9	99.6	100.0
fwer	2000	detection	73.9	89.3	99.6	100.0
uncorr	100	detection	92.9	98.1	100.0	100.0
uncorr	200	detection	93.9	98.3	100.0	100.0
uncorr	400	detection	93.9	98.7	100.0	100.0
uncorr	1000	detection	93.9	98.3	100.0	100.0
uncorr	2000	detection	94.3	98.4	100.0	100.0
bh	100	false_pos	1.0	2.9	9.1	34.3
bh	200	false_pos	1.2	3.0	11.0	37.3
bh	400	false_pos	1.2	3.1	12.2	39.2
bh	1000	false_pos	1.1	3.5	11.8	39.2
bh	2000	false_pos	1.2	3.2	12.2	39.6
fwer	100	false_pos	0.4	1.6	4.8	17.6
fwer	200	false_pos	0.4	1.1	4.2	16.8
fwer	400	false_pos	0.4	1.0	3.7	16.2
fwer	1000	false_pos	0.3	1.0	3.5	15.6
fwer	2000	false_pos	0.3	0.9	3.5	15.9
uncorr	100	false_pos	8.6	14.3	26.4	49.2
uncorr	200	false_pos	8.5	14.2	26.8	50.2
uncorr	400	false_pos	8.2	14.2	27.6	50.5
uncorr	1000	false_pos	8.4	14.2	27.6	50.7
uncorr	2000	false_pos	8.1	14.5	27.5	50.6
bh	100	type_I	0.0	0.0	0.0	0.0
bh	200	type_I	0.0	0.0	0.1	0.2
bh	400	type_I	0.2	0.2	0.2	0.3
bh	1000	type_I	0.2	0.2	0.2	0.3
bh	2000	type_I	0.2	0.2	0.2	0.3
fwer	100	type_I	0.2	0.3	0.4	0.5

rule	B	type	n=150	n=250	n=500	n=1000
fwer	200	type_I	0.2	0.3	0.2	0.4
fwer	400	type_I	0.1	0.2	0.2	0.4
fwer	1000	type_I	0.1	0.2	0.2	0.4
fwer	2000	type_I	0.2	0.2	0.2	0.3
uncorr	100	type_I	4.6	4.8	4.9	4.8
uncorr	200	type_I	4.4	4.7	4.6	4.7
uncorr	400	type_I	4.3	4.6	4.8	4.6
uncorr	1000	type_I	4.2	4.5	4.8	4.6
uncorr	2000	type_I	4.3	4.5	4.7	4.3

Family-wise error rate under a true model

The quantity FWER is meant to bound. BH bounds the false discovery rate instead, so it is not expected to sit at alpha here — the comparison shows what each actually delivers.

```
pv_long |>
  filter(k == 0) |>
  group_by(rule, B, n, rep) |>
  summarise(any = as.integer(sum(flag, na.rm = TRUE) > 0), .groups = "drop") |>
  group_by(rule, B, n) |>
  summarise(v = 100 * mean(any), .groups = "drop") |>
  pivot_wider(names_from = n, values_from = v, names_prefix = "n=") |>
  mutate(across(where(is.numeric), \(x) round(x, 1))) |>
  arrange(rule, B)
```

rule	B	n=150	n=250	n=500	n=1000
bh	100	0.0	0.0	0.0	0.0
bh	200	0.0	0.0	1.2	1.6
bh	400	4.0	3.2	4.0	5.6
bh	1000	2.8	4.0	3.6	5.2
bh	2000	4.0	4.8	4.4	5.2
fwer	100	4.8	6.4	6.8	9.2
fwer	200	3.6	5.2	4.4	7.6
fwer	400	2.4	4.4	3.6	6.8
fwer	1000	2.8	4.0	2.8	7.2
fwer	2000	3.6	3.2	2.8	5.6
uncorr	100	62.4	63.2	64.8	58.4
uncorr	200	59.2	65.6	61.2	58.8
uncorr	400	59.6	62.4	63.6	57.2
uncorr	1000	59.2	62.8	62.4	58.0
uncorr	2000	59.6	62.0	62.0	55.6

Realised false discovery proportion (k = 3)

Among items flagged, the proportion that were in fact fitting. This is what BH targets.

```
pv_long |>
  filter(k == 3, rule %in% c("fwer", "bh")) |>
```

```
group_by(rule, B, n, rep) |>
summarise(flagged = sum(flag, na.rm = TRUE),
           false = sum(flag[!is_misfit], na.rm = TRUE), .groups = "drop") |>
filter(flagged > 0) |>
group_by(rule, B, n) |>
summarise(fdp = round(100 * mean(false / flagged), 1), .groups = "drop") |>
pivot_wider(names_from = n, values_from = fdp, names_prefix = "n=") |>
arrange(rule, B)
```

rule	B	n=150	n=250	n=500	n=1000
bh	100	28.4	30.8	36.3	63.0
bh	200	6.2	11.8	32.0	65.5
bh	400	5.6	11.7	34.9	67.1
bh	1000	4.9	12.7	34.5	66.7
bh	2000	5.4	12.2	35.1	67.2
fw	100	2.2	6.8	17.4	46.0
fw	200	1.8	4.8	15.9	44.1
fw	400	1.9	4.3	14.2	43.2
fw	1000	1.3	4.1	13.6	42.9
fw	2000	1.4	4.1	13.4	42.9

The BH reachability floor

The smallest attainable BH-adjusted p-value is $m / (B + 1)$ for m comparisons, so BH cannot flag a single item unless $B \geq m/\alpha - 1$. With $m = 20$ and $\alpha = .05$ that is $B \geq 399$. FWER's studentised-max p-value has no such floor — it needs only $B \geq 1/\alpha - 1 = 19$.

```
data.frame(B = B2) |>
mutate(marginal_floor = round(1 / (B + 1), 5),
       bh_min_adj_p = round(20 / (B + 1), 4),
       bh_can_flag_one_item = 20 / (B + 1) < ALPHA)
```

B	marginal_floor	bh_min_adj_p	bh_can_flag_one_item
100	0.00990	0.1980	FALSE
200	0.00498	0.0995	FALSE
400	0.00249	0.0499	TRUE
1000	0.00100	0.0200	TRUE
2000	0.00050	0.0100	TRUE

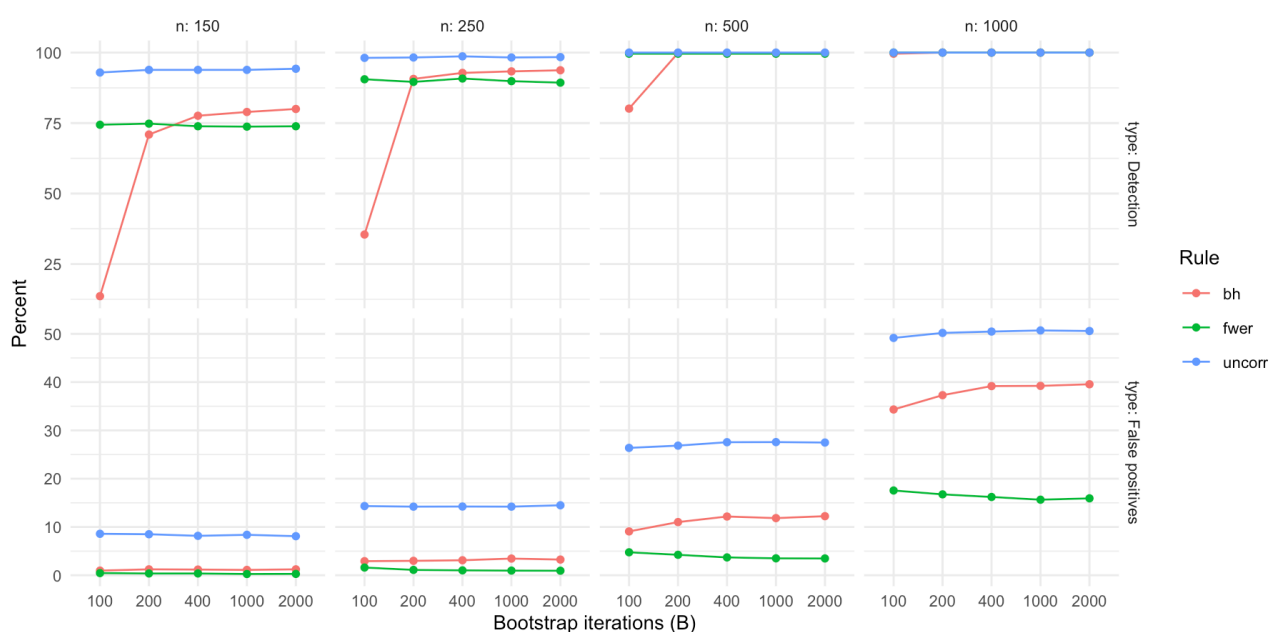
Observed minima, as a check:

```
pval2 |>
group_by(B) |>
summarise(min_p_marg = round(min(p_marg, na.rm = TRUE), 5),
           min_padj_fwer = round(min(padj_fwer, na.rm = TRUE), 5),
           min_padj_bh = round(min(padj_bh, na.rm = TRUE), 5),
           .groups = "drop")
```

B	min_p_marg	min_padj_fwer	min_padj_bh
100	0.00990	0.00990	0.01800
200	0.00498	0.00498	0.00995

B	min_p_marg	min_padj_fwer	min_padj_bh
400	0.00249	0.00249	0.00554
1000	0.00100	0.00100	0.00250
2000	0.00050	0.00050	0.00167

```
pv_long |>
  filter(k == 3) |>
  mutate(type = ifelse(is_misfit, "Detection", "False positives")) |>
  group_by(rule, B, n, type) |>
  summarise(v = 100 * mean(flag, na.rm = TRUE), .groups = "drop") |>
  ggplot(aes(factor(B), v, colour = rule, group = rule)) +
  geom_line() + geom_point() +
  facet_grid(type ~ n, scales = "free_y", labeller = label_both) +
  labs(x = "Bootstrap iterations (B)", y = "Percent", colour = "Rule") +
  theme_minimal(base_size = 10)
```



Notes

Self-contained apart from exported easyRasch2 functions; relative paths throughout, so the whole set can be relocated to a reproducibility folder without modification.